

Building Context-Aware Chatbots with Streamlit and Buffer Memory

October 14, 2024

Introduction

In this lab, we will explore how to build a context-aware chatbot using LangChain, buffer memory, chains, and Streamlit. This lab will guide you through setting up the project, running the chatbot, and experimenting with different personas, language models, and prompts. LangChain allows you to easily build and customize workflows that involve large language models while using memory and chaining actions together. Streamlit helps us build an easy-to-use web interface to interact with the chatbot. We will look into:

- What LangChain, buffer memory, and chains are.
- How to use Streamlit to build an interface for the chatbot.
- How to run the chatbot locally.
- How to customize the chatbot's persona, experiment with different LLMs, and use advanced prompting techniques like Chain of Thought (CoT) and Recursive Criticizing and Improving (RCI).

1 Instructions

1.1 Clone the Repository

The first step is to clone the LangChain playground repository. This repository contains all the code and resources you'll need for this lab. Use the following command to clone it:

```
git clone https://github.com/iafarhan/langchain-playground
```

Once the repository is cloned, navigate to the project directory:

```
cd langchain-playground
```

1.2 Install Required Dependencies

The project uses various Python libraries that need to be installed. To do this, install the dependencies listed in the `requirements.txt` file by running the following command:

```
pip install -r requirements.txt
```

1.3 Create an `.env` File

Create a `.env` file in the root directory to store your API key securely. This file will allow the chatbot to interact with OpenAI's language models. To create the file, use the following command:

```
touch .env
```

Next, open the `.env` file in a text editor and add your OpenAI API key:

```
OPENAI_API_KEY=sk-iv7te4Mr3V4YHC3t5QC4T3B1bkFJpM32BsAeFNEPkLe6w3rB
```

Important: You can get your own API key from [OpenAI Playground](#).

1.4 Run the Streamlit App

Once everything is set up, run the Streamlit application using the following command:

```
streamlit run Home.py
```

This will start the chatbot interface, and you can access it in your browser at <http://localhost:8501>.

2 Understanding Key Components

2.1 LangChain

[LangChain](#) is a framework designed for developing applications powered by large language models (LLMs). It simplifies the development process by providing open-source building blocks, components, and third-party integrations. LangChain allows developers to build complex workflows by chaining prompts, outputs, and functionalities in a modular way. The framework also supports advanced concepts like stateful agents and multi-actor applications through LangGraph, and it offers monitoring and debugging capabilities through LangSmith.

2.2 Buffer Memory

[Buffer memory](#) in LangChain provides conversational memory for chatbots, storing past conversations to maintain context across multiple interactions. Specifically, LangChain supports *conversation buffer window memory*, which acts as short-term memory by remembering only a few recent exchanges. The number of remembered conversations is controlled by a parameter K . For instance, if K is set to 2, it will retain only the last 2 conversations. This is useful for keeping responses relevant while reducing memory load, especially in applications where remembering the entire conversation history is unnecessary.

2.3 Chains

[Chains](#) in LangChain refer to sequences of connected operations that handle user input, processing it through a series of steps like a language model, memory, or external API calls, and then returning the final response. This modular structure allows for the creation of complex workflows where multiple models and tools can interact seamlessly.

2.4 Streamlit

[Streamlit](#) is an open-source framework designed to quickly build interactive web applications for data science and machine learning. In this lab, Streamlit is used to create an easy-to-use, real-time interface for interacting with the chatbot built using LangChain. It allows developers to deploy AI applications with minimal setup and provides a clean interface for users to engage with the chatbot.

3 Tasks

Now that you've set up the chatbot, experiment with the following tasks to enhance your understanding of how LangChain, buffer memory, and chains work.

3.1 Task 1: Change the Persona

Currently, the chatbot is acting as a pirate. Let's change its persona! Open the `Home.py` file and modify the persona by changing the `PromptTemplate`.

Change the persona from *pirate* to a persona of your choice.

```
template = """ You are a(n) adjective pirate having a conversation.
Current conversation:
history
Human: input
AI: """
```

You can also try different personas like:

- A motivational speaker.
- A tech entrepreneur.
- A curious child.

3.2 Task 2: Use Different LLMs

By default, the chatbot is using OpenAI's GPT-4 model. LangChain supports other LLMs like those from Hugging Face. Try using a different LLM by modifying the `utils.configure_llm()` function.

Refer to the LangChain documentation to learn how to integrate models from other providers. You can find the official LangChain guide on working with different LLMs here: [LangChain LLM Integration Documentation](#).

If you'd like to explore models available on Hugging Face, visit their official website: [Hugging Face Models](#).

3.3 Task 3: Experiment with Chain of Thought (CoT) and RCI

Chain of Thought (CoT): This technique allows the model to break down complex tasks into smaller steps, which can improve the accuracy and quality of its responses. You can try using prompts like:

```
"Break down the process of training a neural network step by step."
```

Recursive Criticizing and Improving (RCI): In RCI, the model critiques its own output and then improves it. You can prompt the model to evaluate its previous response and suggest improvements. For example, use a prompt like:

```
"Critique the answer you just gave and provide a better version."
```

3.4 Task 4: Use NeuralWriter to Write Better Prompts

[NeuralWriter](#) helps generate high-quality prompts for language models. Use NeuralWriter to refine and improve the prompts you're sending to the chatbot. Here are a few examples:

- Act as a lawyer and explain the concept of intellectual property to a 5th grader.
- You are a chef. Provide a recipe for a 3-course meal using ingredients available in space.

Try experimenting with NeuralWriter to observe how different prompt structures can affect the LLM's responses.

By following this lab, you have set up a chatbot using LangChain and Streamlit, learned about buffer memory, and explored how to customize chatbot personas and use advanced prompting techniques. These tasks will help you understand how to create more intelligent and context-aware conversational agents.